

ApexPlanet

Cybersecurity & Ethical Hacking Internship

TASK 3 — WEB APPLICATION SECURITY

Security Testing Report with Practical Evidence

Testing Environment: Kali Linux / DVWA local lab

Scope: Controlled local application testing only

Prepared as an internship deliverable

1. Executive Summary

This report documents the Task 3 web-application security exercises completed in a controlled local DVWA environment. The work covers SQL Injection, Reflected and Stored XSS, CSRF, Burp Suite request interception and testing workflow, File Inclusion/LFI, security-header analysis, and local Apache security-header configuration. Screenshots are included as practical evidence.

Important scope note: Testing shown in this report was performed against the local lab environment. No real-world application was targeted or claimed as an exploitation target.

2. Objectives

- Identify common web-application vulnerabilities in a deliberately vulnerable training application.
- Demonstrate controlled SQL Injection and XSS scenarios.
- Demonstrate CSRF password-change behavior and document token-based protection as the mitigation approach.
- Use Burp Suite Proxy, Repeater, and Intruder for controlled request analysis.
- Demonstrate Local File Inclusion and document the RFI limitation observed in the lab.
- Review security headers and apply defensive headers in local Apache configuration.

3. Environment & Scope

Application: DVWA (Damn Vulnerable Web Application)

Execution: Kali Linux local environment

Target scope: 127.0.0.1 / local DVWA instance

Testing level: DVWA Low where required to demonstrate vulnerable behavior

Tools: Burp Suite, browser, Apache, curl, and DVWA modules

4. SQL Injection

Baseline behavior was first checked with a normal input. A basic SQL injection test was then performed in DVWA's SQL Injection module. The purpose was to demonstrate how insufficiently protected database queries can interpret user-controlled input as SQL.

Mitigation / Defensive Notes: Mitigation: use parameterized/prepared statements, validate input, apply least-privilege database accounts, and avoid constructing SQL queries by string concatenation.

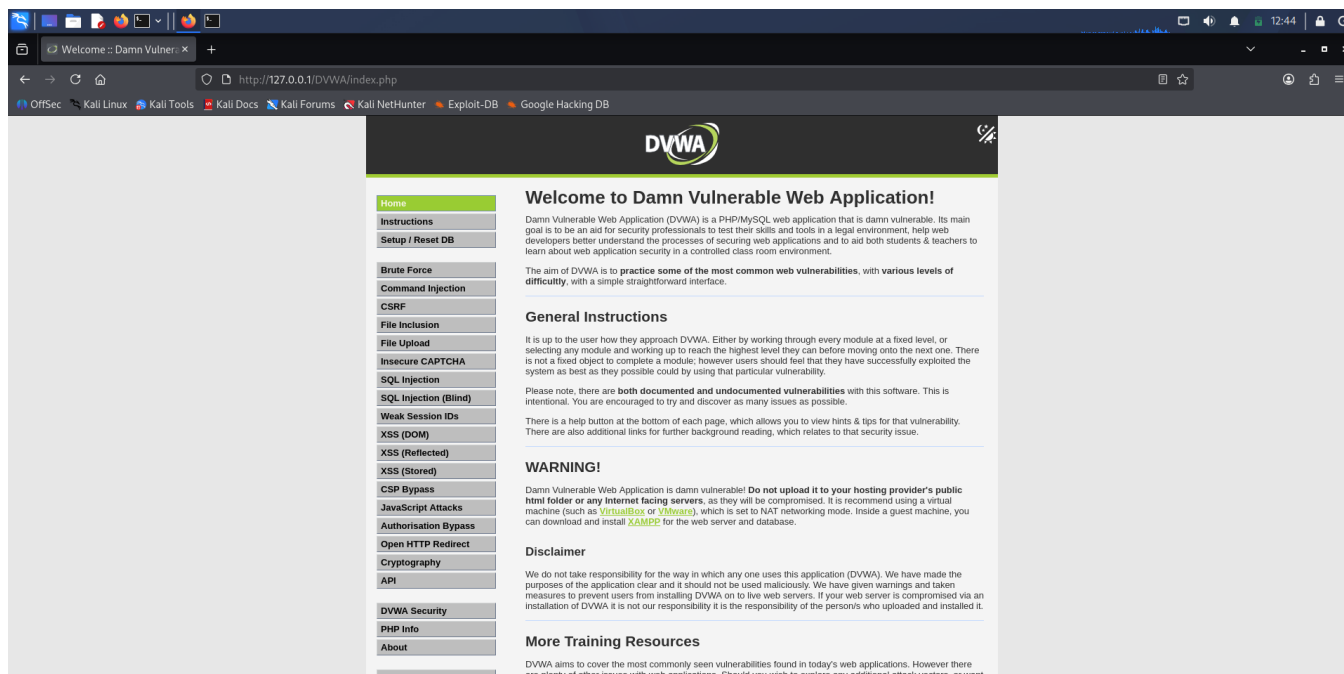


Figure: DVWA Dashboard — DVWA was accessed in the local lab environment.

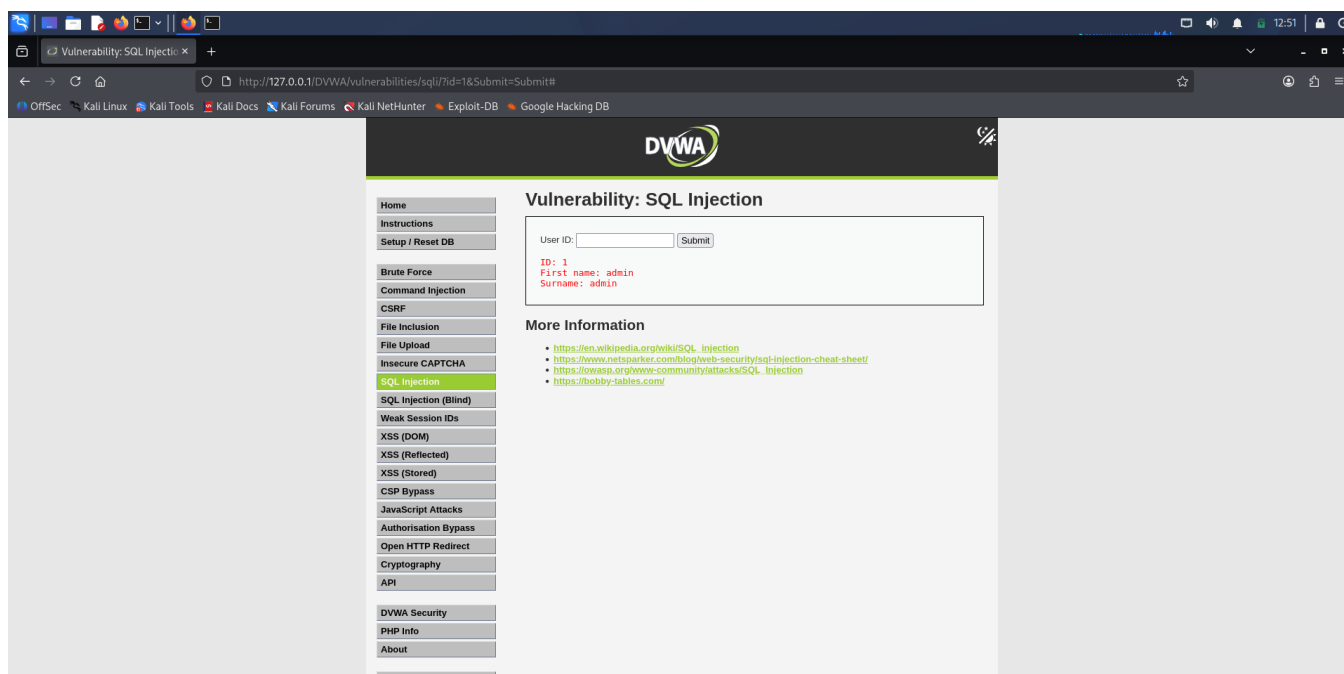


Figure: SQL Injection — Normal Input — A normal numeric input was tested first to establish baseline behavior.

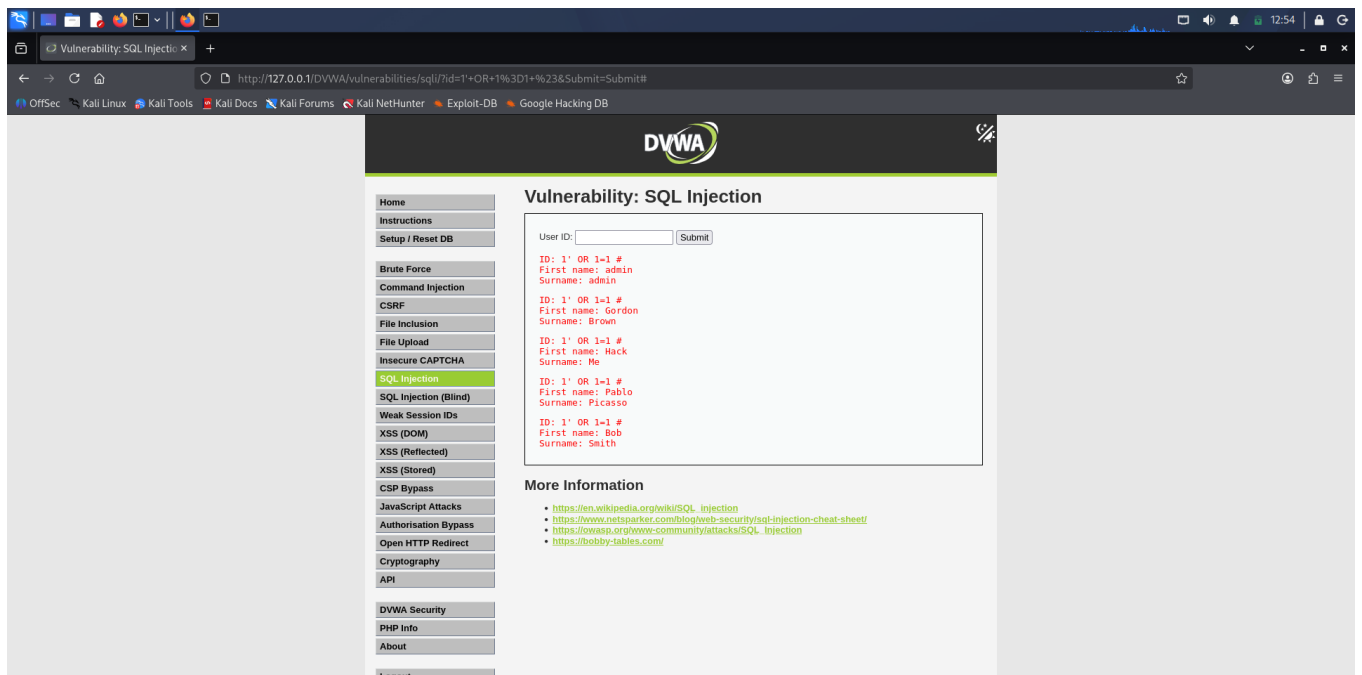


Figure: SQL Injection — Basic Injection — A basic SQL injection test was demonstrated in DVWA's controlled SQL Injection module.

5. Cross-Site Scripting (XSS)

Both reflected and stored XSS scenarios were demonstrated in the local DVWA application. Reflected input is returned in the immediate response, while stored input can persist in application data and affect later viewers.

Mitigation / Defensive Notes: Mitigation: context-aware output encoding, strict input validation, safe templating, and an appropriately configured Content Security Policy (CSP).

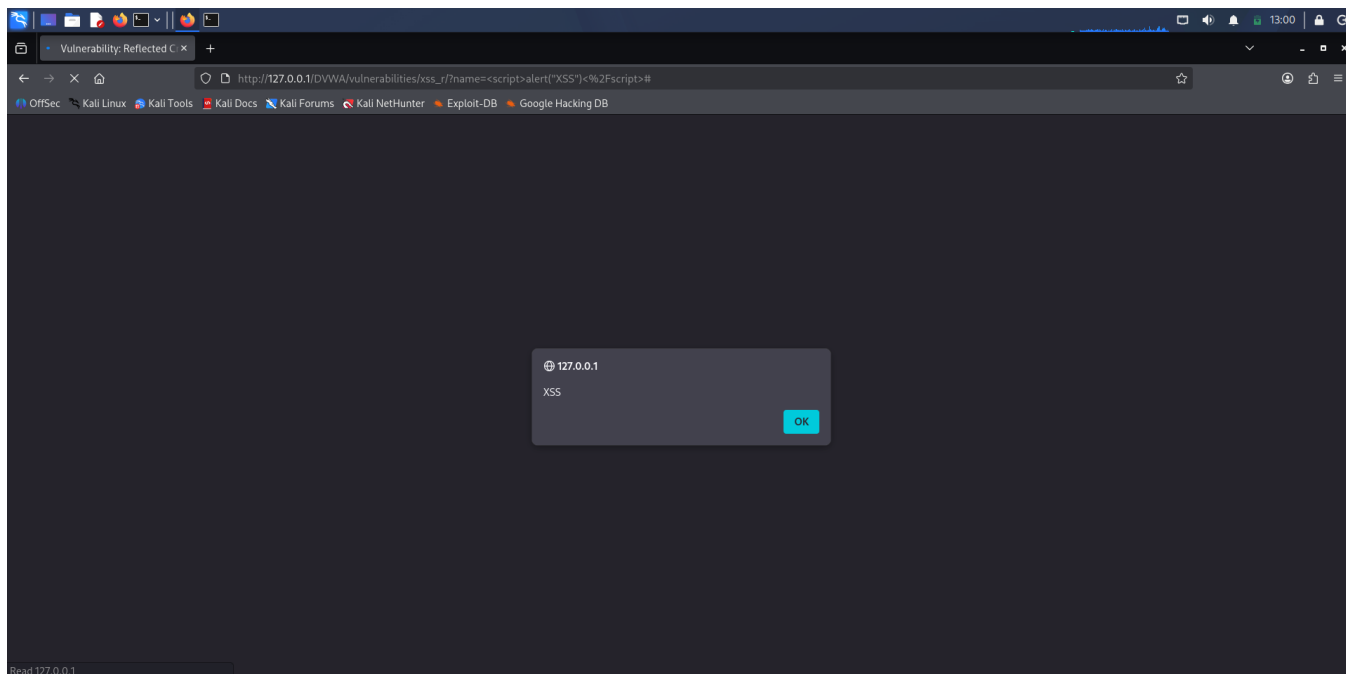


Figure: Reflected XSS — Reflected XSS behavior was demonstrated in the local DVWA lab.

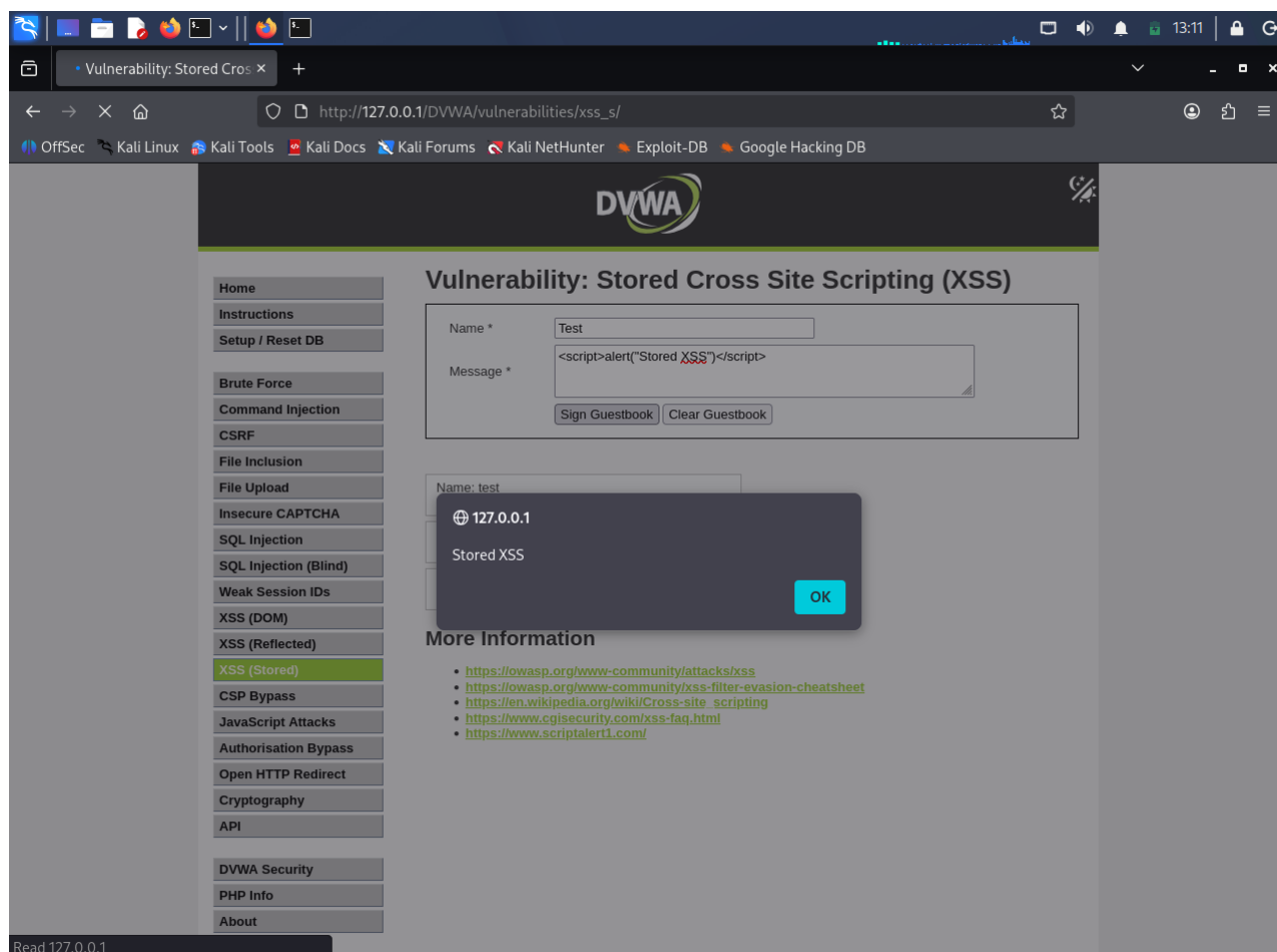


Figure: Stored XSS — Stored XSS behavior was demonstrated in the local DVWA lab.

6. Cross-Site Request Forgery (CSRF)

The local DVWA CSRF scenario was used to demonstrate a password-change request. The screenshot records the successful controlled lab outcome.

Mitigation / Defensive Notes: Mitigation: use unpredictable anti-CSRF tokens tied to the user's session, validate request origin where appropriate, and use suitable SameSite cookie settings.

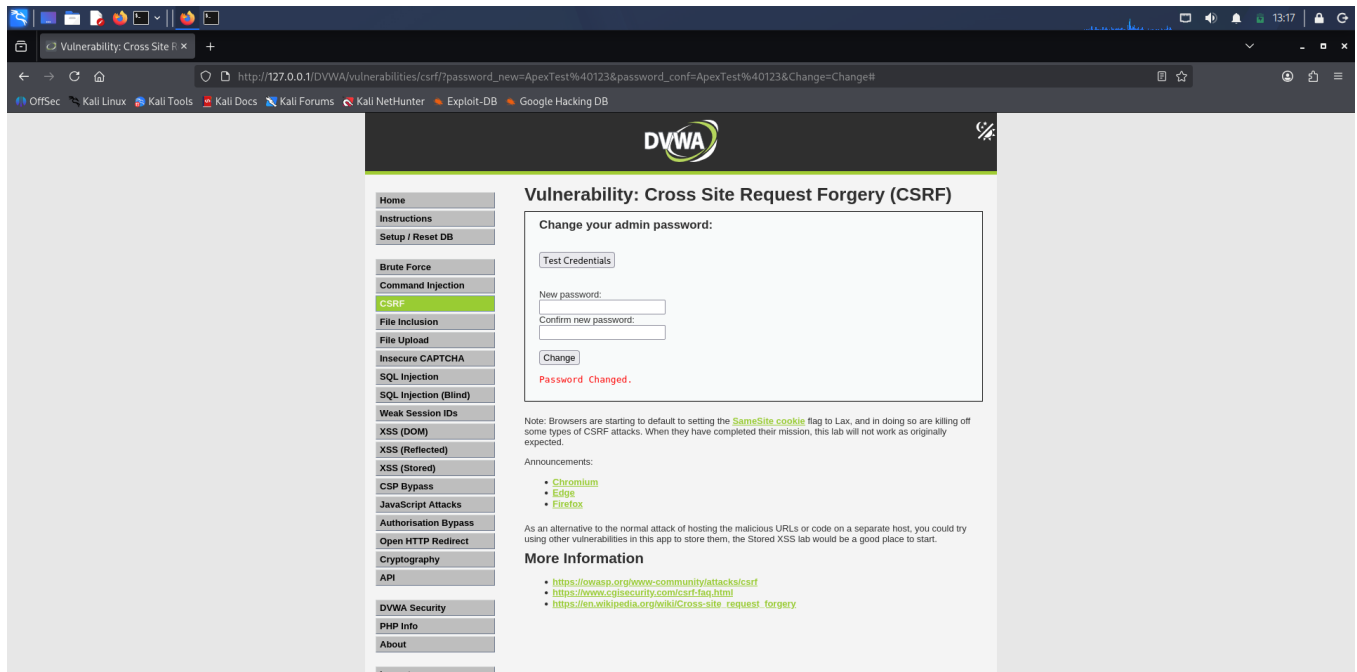


Figure: CSRF — Password Change — The CSRF password-change scenario was demonstrated in the controlled DVWA environment.

7. Burp Suite — Proxy, Repeater & Intruder

Burp Suite was used as an intercepting proxy for the local DVWA application. A request was intercepted, sent to Repeater for controlled inspection, and then loaded into Intruder to document the request-fuzzing workflow.

Mitigation / Defensive Notes: Mitigation/defense: validate and authorize server-side requests regardless of client-side manipulation; apply rate limiting and strong input validation where relevant.

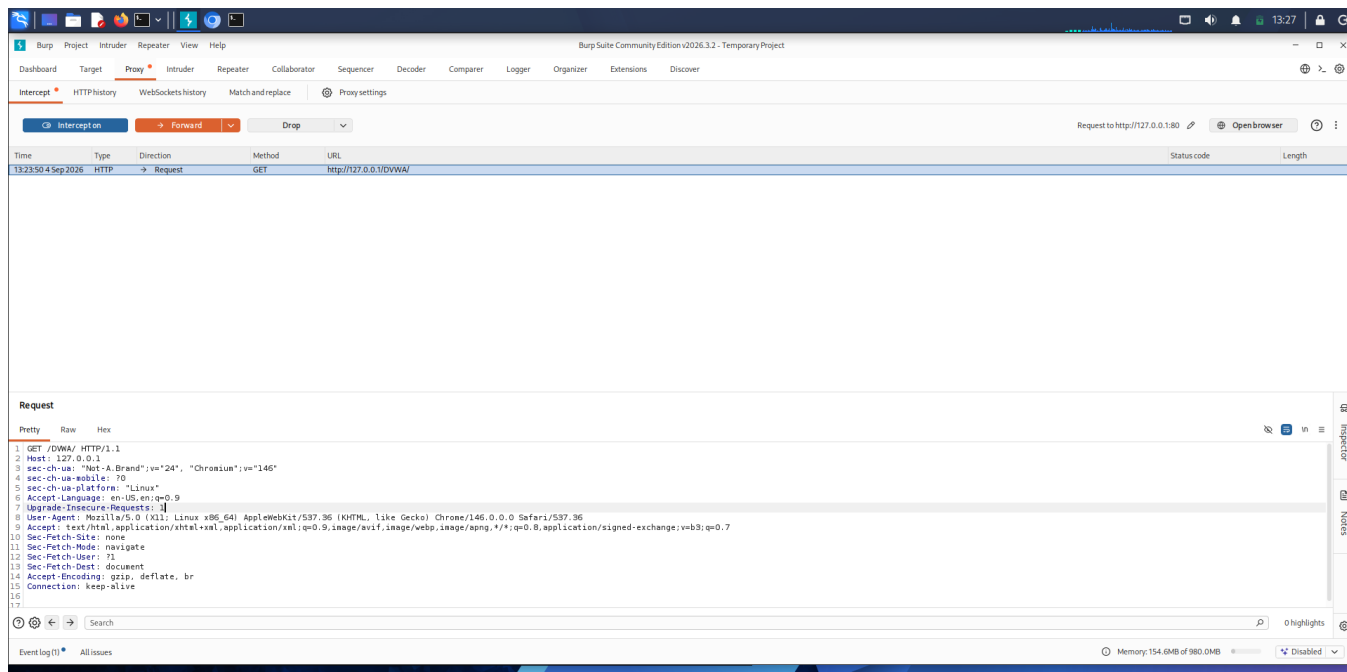


Figure: Burp Suite — Intercepted Request — Burp Suite Proxy interception was used to observe a local DVWA request.

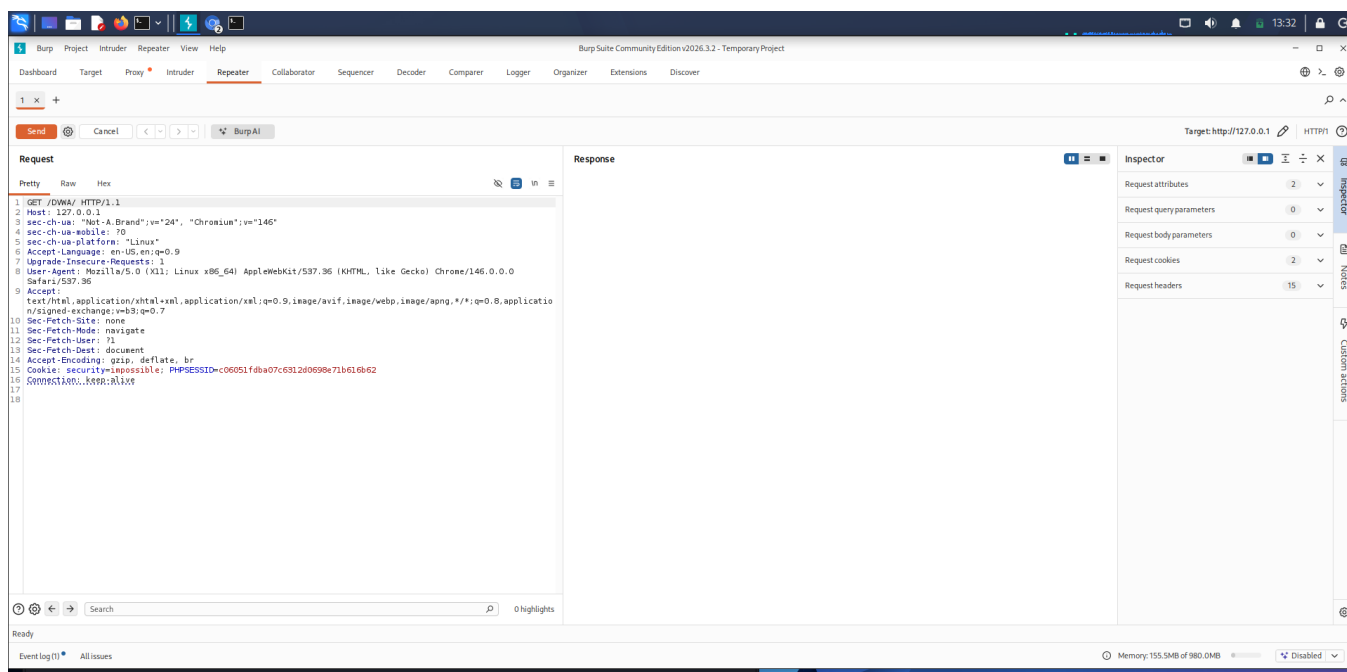


Figure: Burp Suite — Repeater — The captured local request was sent to Repeater for controlled request inspection.

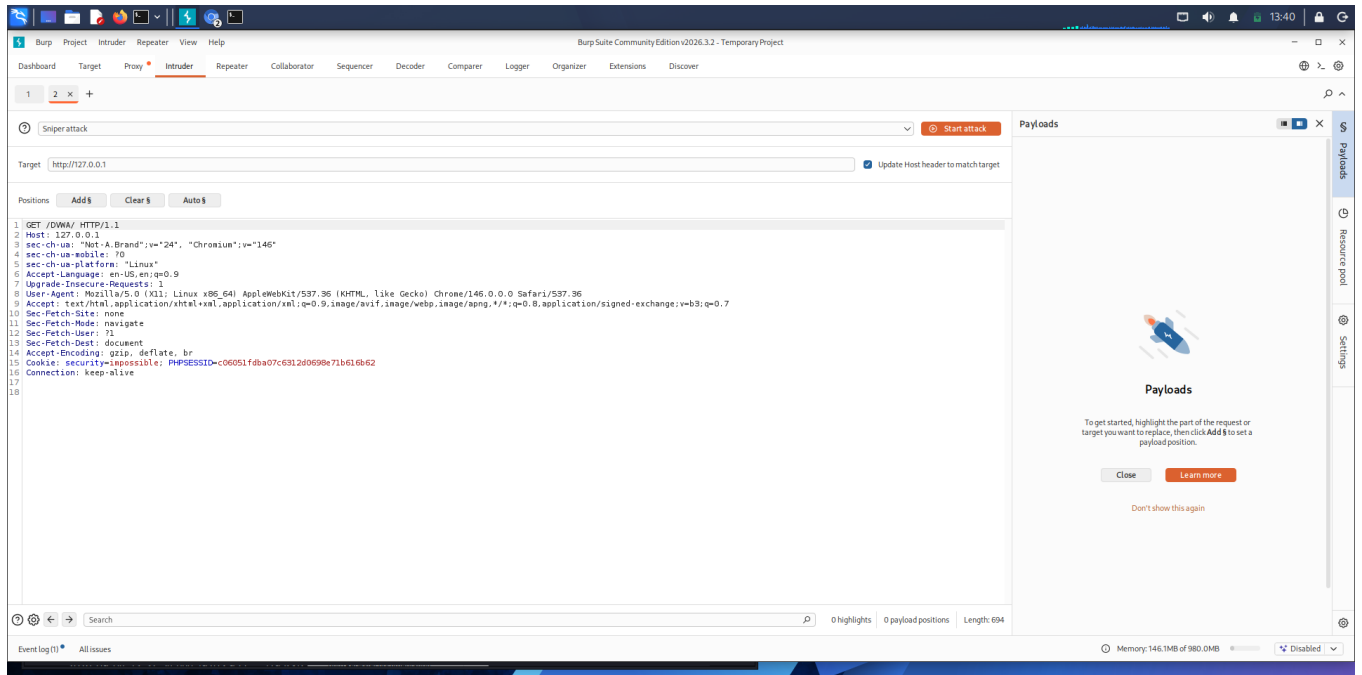


Figure: Burp Suite — Intruder — The local request was loaded into Intruder to document the fuzzing workflow.

8. File Inclusion & LFI

DVWA's File Inclusion module was accessed and a Local File Inclusion test was demonstrated against the local application. The module also displayed that URL-based inclusion was not enabled in the current configuration.

Mitigation / Defensive Notes: Mitigation: avoid using raw user input as file paths, use allowlists of permitted files, normalize paths, apply least privilege, and disable unnecessary remote inclusion functionality.

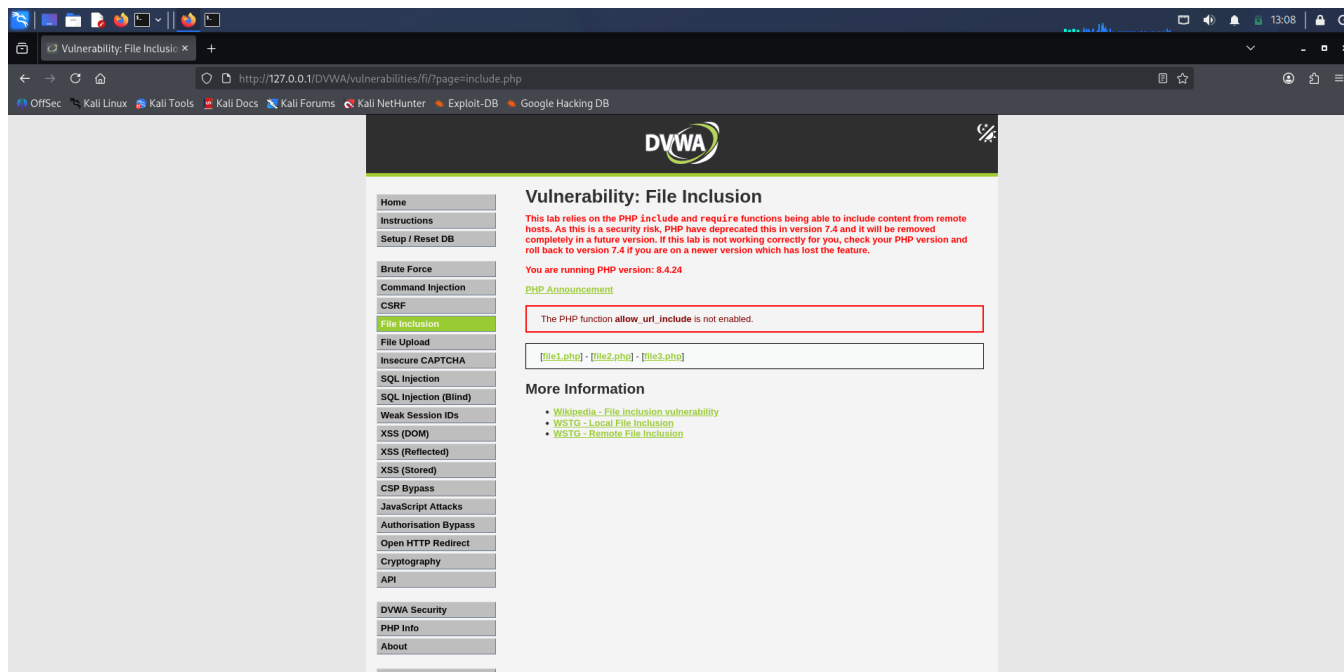


Figure: File Inclusion Module — DVWA File Inclusion was accessed. The lab displayed the local file links and the URL-include warning.

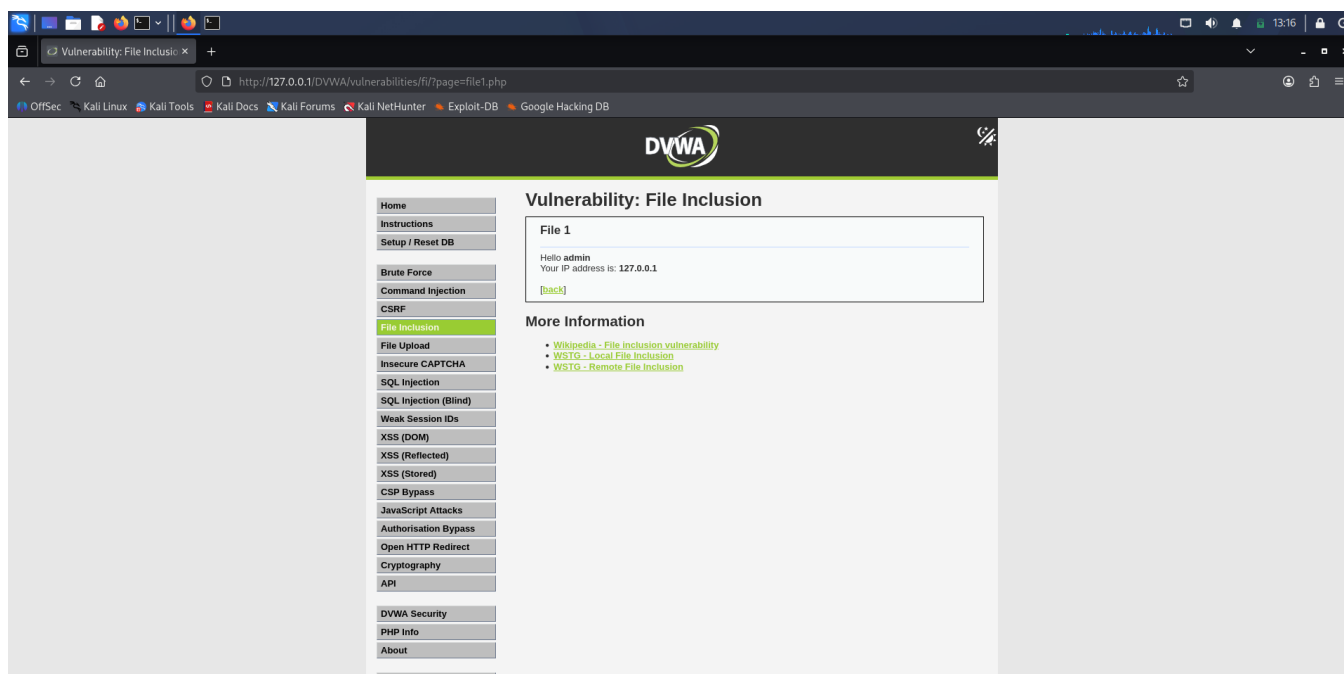
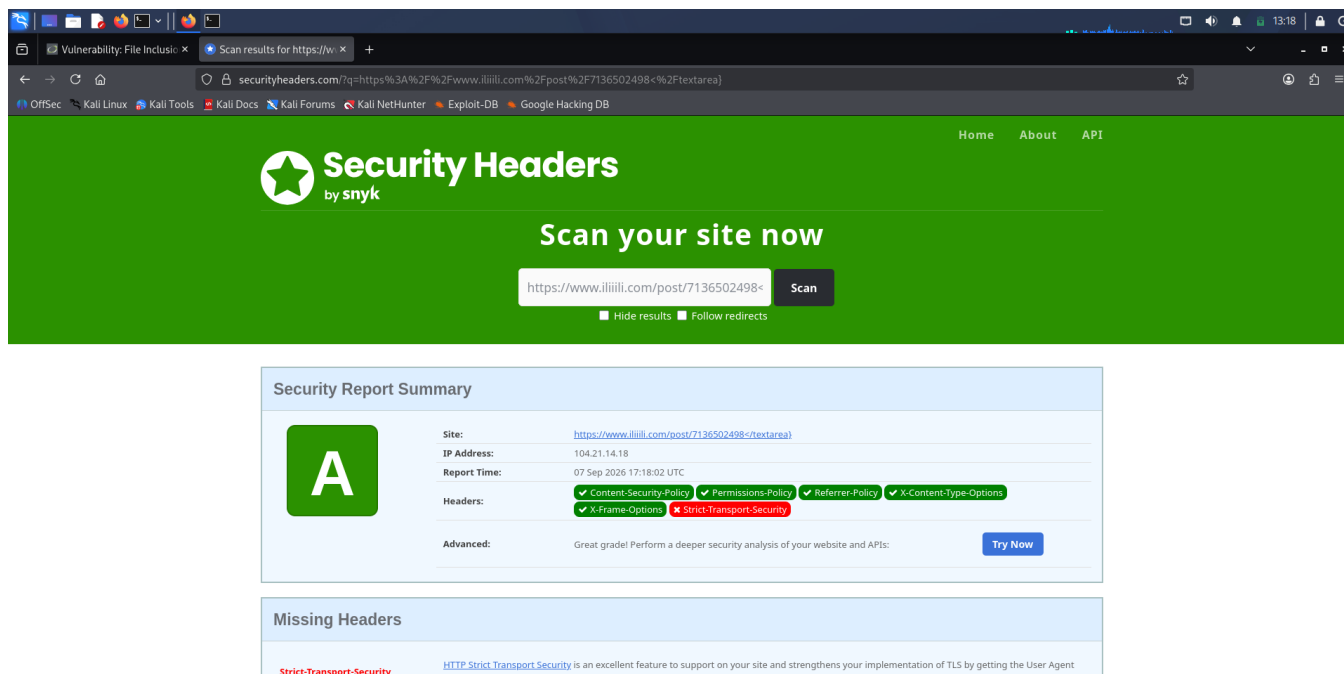


Figure: Local File Inclusion (LFI) — A local-file inclusion test was demonstrated against the local DVWA instance.

9. Security Headers

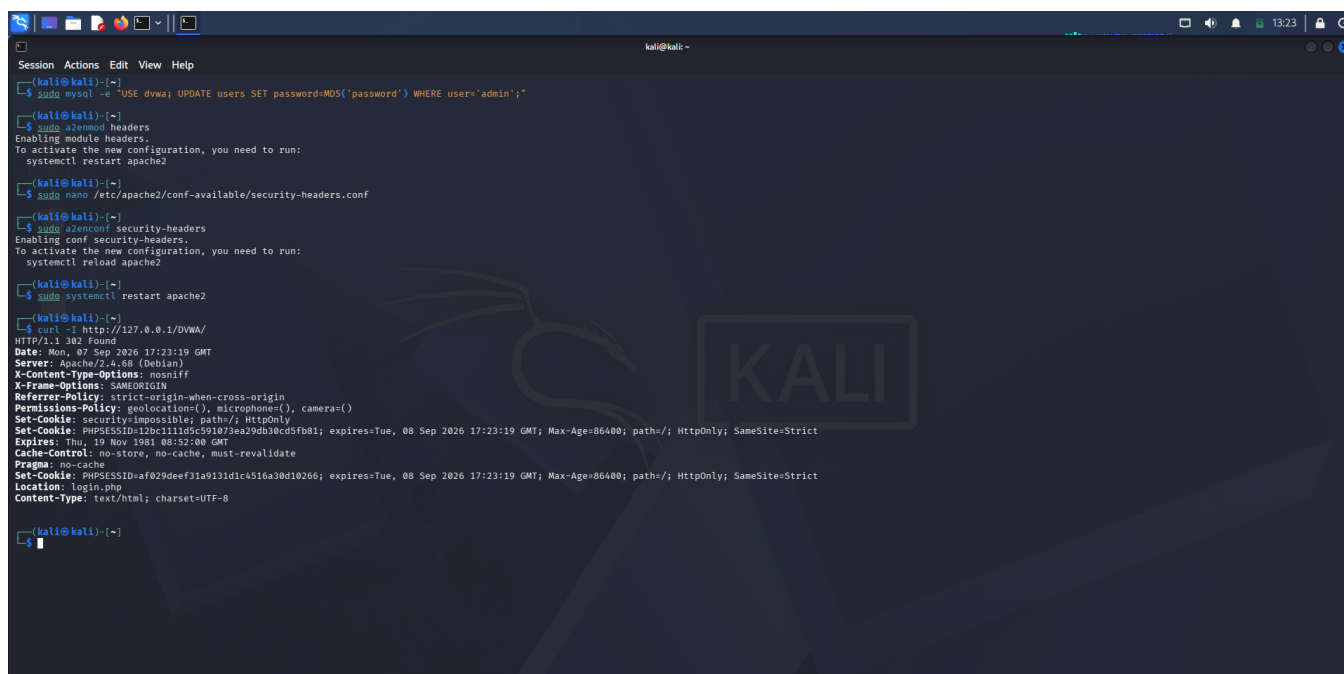
A security-header analysis was recorded for a test site, followed by defensive configuration of security headers in the local Apache server. The final screenshot verifies the configured response headers.

Mitigation / Defensive Notes: Headers applied in the local Apache configuration included X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy. A CSP should also be designed and deployed according to the application's actual resource requirements.



The screenshot shows the Security Headers website interface. At the top, there's a navigation bar with 'Home', 'About', and 'API' links. The main heading is 'Security Headers by snyk'. Below it, a large green button says 'Scan your site now'. A text input field contains the URL 'https://www.iliilli.com/post/7136502498<'. A 'Scan' button is next to it. Below the input field, there are checkboxes for 'Hide results' and 'Follow redirects'. The scan results are displayed in a 'Security Report Summary' section. It shows a large green 'A' grade. The site URL is 'https://www.iliilli.com/post/7136502498<'. The IP address is '104.21.14.18'. The report time is '07 Sep 2026 17:18:02 UTC'. The headers section shows a list of headers: 'Content-Security-Policy', 'Permissions-Policy', 'Referrer-Policy', 'X-Content-Type-Options', 'X-Frame-Options', and 'Strict-Transport-Security'. The 'Strict-Transport-Security' header is highlighted in red, indicating it is missing. The 'Advanced' section says 'Great grade! Perform a deeper security analysis of your website and APIs:' with a 'Try Now' button. Below the summary, there's a 'Missing Headers' section. It lists 'Strict-Transport-Security' as missing and provides a link to 'HTTP Strict Transport Security' with a description: 'HTTP Strict Transport Security is an excellent feature to support on your site and strengthens your implementation of TLS by getting the User Agent to enforce the use of HTTPS. Recommended value: "Strict-Transport-Security: max-age=31536000; includeSubDomains;"

Figure: Security Headers Analysis — A security-header analysis of a test site was recorded as part of the assessment workflow.



The screenshot shows a terminal window on a Kali Linux machine. The user is in the root directory and runs the following commands:
1. `sudo mysql -e "USE dwaa; UPDATE users SET password=MD5('password') WHERE user='admin';"`
2. `sudo a2enmod headers`
3. `sudo nano /etc/apache2/conf-available/security-headers.conf`
4. `sudo a2enconf security-headers`
5. `sudo systemctl restart apache2`
6. `curl -I http://127.0.0.1/DVWA/`
The output of the curl command shows the following headers:
HTTP/1.1 302 Found
Date: Mon, 07 Sep 2026 17:23:19 GMT
Server: Apache/2.4.68 (Debian)
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=af029deef31a9131d1c4516a30d10266; expires=Tue, 08 Sep 2026 17:23:19 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=af029deef31a9131d1c4516a30d10266; expires=Tue, 08 Sep 2026 17:23:19 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Location: login.php
Content-Type: text/html; charset=UTF-8

Figure: Apache Security Headers — Verification — Security headers configured in the local Apache server were verified with an HTTP response check.

10. RFI Limitation Observed

The DVWA File Inclusion module displayed a warning that URL-based inclusion was not enabled in the current PHP configuration. Therefore, no RFI execution result is claimed in this report. This is documented as a lab/configuration limitation rather than presenting a fabricated result.

11. Findings Summary

Area	Demonstration	Primary mitigation
SQL Injection	Controlled DVWA SQLi test	Prepared/parameterized statements
Reflected XSS	Local reflected XSS behavior	Output encoding + validation + CSP
Stored XSS	Local stored XSS behavior	Output encoding + validation + CSP
CSRF	Controlled password-change scenario	Anti-CSRF token + SameSite controls
Burp Suite	Proxy → Repeater → Intruder workflow	Server-side validation/authorization
LFI	Local file inclusion test	File allowlist + path validation
RFI	Not executed; disabled in lab	Disable remote inclusion unless explicitly required
Security Headers	Analysis + local Apache verification	Defensive HTTP security headers

12. Evidence Index

The supplied evidence archive contains the screenshots listed below. The duplicate typo-named file *04-XSS-Reflecteed.png* was not used in the report because the correctly named *04-XSS-Reflected.png* is present.

- 01-DVWA-Dashboard.png** — DVWA Dashboard
- 02-SQLi-Normal-Input.png** — SQL Injection — Normal Input
- 03-SQLi-Basic-Injection.png** — SQL Injection — Basic Injection
- 04-XSS-Reflected.png** — Reflected XSS
- 05-XSS-Stored.png** — Stored XSS
- 06-CSRF-Password-Changed.png** — CSRF — Password Change
- 07-Burp-Intercepted-Request.png** — Burp Suite — Intercepted Request
- 08-Burp-Repeater-Request.png** — Burp Suite — Repeater
- 09-Burp-Intruder.png** — Burp Suite — Intruder
- 10-File-Inclusion-Page.png** — File Inclusion Module
- 11-LFI-Local-File.png** — Local File Inclusion (LFI)
- 12-Security-Headers-Analysis.png** — Security Headers Analysis
- 13-Apache-Security-Headers.png** — Apache Security Headers — Verification

13. Conclusion

Task 3 practical web-application security exercises were documented with screenshots from the controlled DVWA lab. The report records both successful demonstrations and the RFI configuration limitation observed during testing. The defensive section emphasizes prepared statements, input/output handling, CSRF tokens, server-side authorization, safe file handling, and HTTP security headers.